Slide 1 — Week 2

Relational Design and Normalization

Topics:

- Mapping ER → relational schema
- Primary keys and foreign keys
- Integrity constraints
- Data anomalies and functional dependencies
- Normalization: 1NF, 2NF, 3NF, BCNF (and 4NF)

Slide 2 — From Conceptual Model to Tables

Last week:

ER Diagram

This week:

Convert ER model → relational tables.

Slide 3 — Example ER Model

Entities:

Customers

Products

Orders

Payments

Goal:

Convert these into SQL tables.

Slide 4 — Entity → Table

Each entity becomes a table.

Example:

Customers → customers table

Attributes → columns

Slide 5 — Example Table

```sql
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(200),
  country VARCHAR(50)
);
```

Slide 6 — Primary Keys

Primary keys:

- uniquely identify rows
- cannot be NULL

Example:

customer_id

Slide 7 — Foreign Keys

Foreign keys link tables together.

Example:

orders.customer_id → customers.customer_id

Slide 8 — Example Orders Table

```
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  order_date DATE,
  FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);
```

Slide 9 — Many-to-Many Relationships

Example:

Orders contain multiple products.

Products appear in many orders.

Solution:

Use a **junction table**.

Slide 10 — Order Items Table

```sql
CREATE TABLE order_items (
  order_item_id INT PRIMARY KEY,
  order_id INT,
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id)
    REFERENCES orders(order_id),
  FOREIGN KEY (product_id)
    REFERENCES products(product_id)
);
```

Slide 11 — Integrity Constraints

Common constraints:

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

CHECK

Example:

```
price DECIMAL(10,2) NOT NULL CHECK (price >= 0)
```

These ensure data quality.

Slide 12 — Data Anomalies

When one table does the job of several entities, three things go wrong:

- **Insert anomaly** — can't add a fact without an unrelated fact attached
- **Update anomaly** — a repeated fact is changed in one row but not the others
- **Delete anomaly** — deleting one fact accidentally erases another

Normalization removes the cause.

Slide 13 — Functional Dependencies

`X → Y` ("X determines Y"): any two rows that agree on X must agree on Y.

Examples:

- order_id → order_date
- product_id → product_name, price
- customer_id → customer_email

A functional dependency is a **business rule**, not a pattern you spot in a few sample rows.

Slide 14 — What is Normalization?

Normalization improves database structure.

Goals:

- reduce redundancy
- prevent anomalies
- improve consistency

Slide 15 — First Normal Form (1NF)

1NF requirements:

- atomic values
- no repeating groups

Bad design:

multiple values in one column.

Slide 16 — Example of 1NF Violation

Bad table:

Orders

order_id | products
1 | laptop, mouse

Better:

Use order_items table.

Slide 17 — Second Normal Form (2NF)

Requirements:

- table must be in 1NF
- remove partial dependency (on part of a composite key)

Example:

order_items(order_id, product_id, quantity, price), key (order_id, product_id)

price depends on product_id alone → move it to products.

Slide 18 — Third Normal Form (3NF)

Requirements:

- table must be in 2NF
- remove transitive dependency (non-key → non-key)

Example:

orders(order_id, customer_id, customer_email)

order_id → customer_id → customer_email

Email belongs to the customer, not the order → move it to customers.

Slide 19 — Boyce–Codd Normal Form (BCNF)

Rule: every determinant must be a superkey.

Example: advising(student_id, subject, instructor)

- a student has one instructor per subject
- each instructor teaches exactly one subject

instructor → subject, but instructor is not a key → BCNF violated.

Fix: instructors(instructor, subject) + student_instructor(student_id, instructor)

Trade-off: BCNF can lose a dependency that 3NF would keep.

Slide 20 — A Note on 4NF

Two **independent** multivalued facts in one table:

claim_id | document_type | adjuster

2 document types × 2 adjusters = 4 rows of every combination.

Fix: split into

claim_documents(claim_id, document_type)
claim_adjusters(claim_id, adjuster)

Slide 21 — Benefits of Normalization

Normalization:

- reduces data duplication
- improves update consistency
- simplifies maintenance

Slide 22 — Denormalization

Sometimes systems intentionally denormalize.

Reasons:

- performance
- analytics workloads

But OLTP systems usually prefer normalized design.

Slide 23 — Relational Design Workflow

1. Design ER diagram
2. Convert entities → tables
3. Define primary keys
4. Define foreign keys
5. Check functional dependencies; normalize to 3NF / BCNF

Slide 24 — Week 2 Labs

Each session has a core lab (**a**) and a challenge lab (**b**), started in the last ~40 minutes of class and submitted by 11:00 pm.

Session 3 — Normalization: 1NF → 3NF → BCNF

- Lab 1a — Larkspur Music School: 1NF, 2NF, 3NF one violation at a time; anomalies
- Lab 1b — Cascade Analytics: testing FDs, closures and candidate keys, BCNF

Session 4 — Keys and constraints; 4NF

- Lab 2a — Ironwood Hardware: PK, FKs, NOT NULL / UNIQUE / CHECK, NULL traps
- Lab 2b — Silverline Mutual: natural vs surrogate keys, a multivalued dependency → 4NF